FULL-STACK SOFTWARE ENGINEERING ROADMAP WITH AI SPECIALIZATION

6-Month Week-by-Week Structured Plan • 5–6 Hours/Day

PHASE 1	PHASE 2	PHASE 3	PHASE 4
Core Foundations	Frontend Eng.	Backend Eng.	AI Integration
Weeks 1–6	*Weeks 7–14*	*Weeks 15–22*	*Weeks 23–27*

Prepared for a CSE Graduate | Dedicated to Building a Serious Engineering Career

Overview & Approach

This roadmap is built for one goal: making you a strong, hireable software engineer with a genuine AI specialization within 6 months. It is structured week-by-week and assumes 5–6 hours of focused work per day. LeetCode is handled separately — this plan covers full-stack engineering only.

Daily Schedule

3 hrs — Concept study + code
2 hrs — Project building / exercises
30 min — Notes + review
Separate — LeetCode (your existing habit)

Core Principles

Fundamentals over frameworks always
Build real projects every week
Push to GitHub every single day
TypeScript from Week 4 onwards

PHASE 1 Core Foundations *Weeks 1–6 | ~5–6 hrs/day*

The goal here is to build the kind of deep fundamentals that separate engineers from code-copiers. Every concept here will be tested in interviews.

Week 1 JavaScript Deep Dive — The Engine & Scope

Focus	JS execution model, call stack, heap, scope, closures, hoisting
Key Topics	Lexical scope, scope chain, closures (references not values), var/let/const, temporal dead zone
Daily Split	3 hrs concept + code, 1.5 hrs exercises, 30 min review/notes
Project	Build 5 small programs that demonstrate closures and scope in non-trivial ways
Interview Prep	Be able to explain: what is a closure? What is hoisting? What is the temporal dead zone?

Week 2 JavaScript Deep Dive — Async & The Event Loop

Focus	Event loop, call stack, Web APIs, callback queue, microtask queue
Key Topics	Why Promises resolve before setTimeout at 0ms, Promise construction & chaining, error propagation, async/await as syntactic sugar
Project	Build a custom Promise implementation from scratch
Interview Prep	Be able to trace execution order for complex async code on a whiteboard

Week 3 JavaScript Deep Dive — OOP, Prototypes & Functional

Focus	Prototype chain, 'this' binding, functional programming
Key Topics	Object.create, class syntax, new keyword, prototype chain; pure functions, immutability, map/filter/reduce from scratch; implicit/explicit/new/arrow 'this' binding
Project	Implement map, filter, reduce, and a custom event emitter from scratch
Interview Prep	Be able to explain all 4 'this' binding rules without hesitation

Week 4 TypeScript Fundamentals

Focus	Type system thinking — structural vs nominal typing

Key Topics	Core types, union/intersection/literal types, generics, utility types (Partial, Required, Pick, Omit, Record), type narrowing, discriminated unions
Mindset	TypeScript is a tool for thinking clearly about your program's data. Always ask: what shape is this data?
Project	Convert a Week 1–3 JS project fully to TypeScript

Week 5 TypeScript Advanced + Tooling

Focus	Production-level TypeScript usage and developer tooling
Key Topics	interface vs type, mapped types, conditional types, template literal types, declaration files (.d.ts), tsconfig.json deep dive
Tooling	ESLint + Prettier + esbuild or tsc build pipeline setup from scratch
Project	Set up a complete TypeScript project from scratch with full tooling config

Week 6 Computer Science Fundamentals Review

Data Structures	Implement linked list, stack, queue, and BST in TypeScript (for understanding, separate from LeetCode)
HTTP	Full request/response lifecycle, status codes, headers, cookies, CORS, HTTPS/TLS handshake
Browser	Parsing HTML/CSS, DOM, CSSOM, render tree, layout, paint, composite pipeline
Networking	DNS, TCP/IP, latency vs bandwidth, CDN fundamentals
Why	This knowledge makes you dangerous in system design and debugging interviews

PHASE 2 Frontend Engineering *Weeks 7–14 | ~5–6 hrs/day*

Build UIs the way professional engineers do — with component architecture, state management, and performance in mind.

Week 7 HTML & CSS Fundamentals — Done Right

Focus	The fundamentals that CS graduates skip and that show in their work
Key Topics	Box model, all positioning types, Flexbox deeply, CSS Grid deeply, specificity, cascade, semantic HTML
Project	Build 3 layouts from scratch: responsive navbar, card grid, and form — no frameworks, no shortcuts
Bonus	Dark mode implementation, accessibility (ARIA labels, keyboard navigation)

Week 8 React Fundamentals — Mental Model First

Focus	React as a declarative library that maps state to DOM
Key Topics	Component tree, reconciliation, JSX, props, composition, useState, useEffect (dependency array as declaration, not trick)
Project	Task manager with full CRUD — local state only, no external libraries
Rule	Understand what React is doing before using what React provides

Week 9 React Advanced Patterns + TypeScript

Focus	How professionals write React
Key Topics	useRef, useCallback, useMemo (when necessary vs premature), custom hooks as primary abstraction, useContext, controlled vs uncontrolled components, compound components
TypeScript	Typing props, events, refs, and custom hooks fully
Project	Build a typed component library: Button, Input, Modal, Dropdown

Week 10 Next.js — App Router Architecture

Focus	How modern React apps are actually built in production
Key Topics	Server vs client components (fundamental to performance), file-based routing, layouts, loading states, error boundaries, data fetching with caching, revalidation

Project	Multi-page application with home, dynamic routes, and an API route — deployed to Vercel
Why Next.js	Industry standard for React apps — not knowing it is a hiring disadvantage

Week 11 State Management + Data Fetching

Zustand	Global client state — understand why preferred over Redux for most apps; solves prop drilling and cross-component state
TanStack Query	Server state management — caching, invalidation, optimistic updates, background refetching
Standard	These are not nice-to-haves. They are the industry standard for modern React apps
Project	Refactor Week 10 project with both Zustand and TanStack Query properly integrated

Week 12 CSS Architecture + Tailwind CSS

Focus	Styling at scale — utility-first philosophy and why it works
Topics	CSS Modules for component scoping, Tailwind CSS deeply — config, custom themes, responsive prefixes
Mindset	Don't memorize classes — internalize the mental model of the system
Project	Fully responsive, dark-mode-enabled UI for your running project using Tailwind

Week 13 Frontend Performance + Testing

Performance	Core Web Vitals (LCP, INP, CLS), code splitting, lazy loading, image/font optimization strategies
Unit Testing	Vitest — unit tests for utility functions and hooks
Component Tests	React Testing Library — test behavior, not implementation
E2E Tests	Playwright basics — critical user flow coverage
Interview Value	Testing philosophy (what to test and why) is a strong signal in interviews

Week 14 Frontend Capstone Project

Project	Developer portfolio + blog — homepage, projects section, MDX blog
Stack	Next.js App Router + TypeScript + Tailwind CSS + TanStack Query

Requirements	Fully responsive, dark mode, deployed, tested, proper README
Goal	Something you are proud to show in interviews — this lives on your GitHub and resume

PHASE 3 Backend Engineering *Weeks 15–22 | ~5–6 hrs/day*

This is where your CS degree pays off. Backend is about systems thinking, not just writing API endpoints.

Week 15 Node.js Deep Dive — Runtime, Not Just Framework

Focus	Node.js as a runtime — libuv, thread pool, non-blocking I/O
Key Topics	Node event loop vs browser event loop, streams and buffers, core modules: fs, path, http, crypto
Project	Build a raw HTTP server without Express — understand what frameworks abstract
Why	Makes you stronger than most bootcamp graduates who only know Express

Week 16 Express.js + REST API Design

Express	Middleware pipeline, routing, error-handling middleware, how next() works
REST Design	Resource naming, HTTP methods, status codes, versioning, pagination
Validation	Zod for runtime input validation — understand why it matters even with TypeScript
Project	Full REST API for a blog platform (users, posts, comments) in TypeScript with Zod

Week 17 Databases — PostgreSQL + Prisma

SQL First	3 days: data types, normalization (1NF–3NF), primary/foreign keys, indexes, ACID transactions — write raw SQL
SQL Practice	Joins, subqueries, aggregations, window functions
Prisma	Schema design, migrations, relations, type-safe queries — understand what it generates and why
Indexing	Which columns to index, composite indexes, EXPLAIN ANALYZE
Project	Connect Week 16 API to PostgreSQL via Prisma with full migrations

Week 18 Authentication + Security

JWT	How tokens are signed and verified, statelessness, revocation problem

Sessions	Session-based auth and when to prefer it over JWT
Password Security	bcrypt hashing — cost factor, why plain storage is career-ending
Vulnerabilities	SQL injection, XSS, CSRF — understand and prevent each
HTTP Security	CORS configuration, security headers (HSTS, CSP, X-Frame-Options)
Project	Complete auth system: registration, login, access + refresh tokens, protected routes

Week 19 Advanced Backend Patterns

Redis	Cache-aside pattern, TTL strategy, session storage, rate limiting with Redis
Job Queues	BullMQ — why you don't do heavy work in the request/response cycle
File Storage	S3-compatible uploads (Cloudflare R2 or MinIO locally)
WebSockets	Socket.io for real-time features
Project	Add one of these to your running project — make it production-quality

Week 20 System Design Fundamentals

Scaling	Load balancing, horizontal vs vertical scaling, statelessness
Databases at Scale	Replication (primary/replica), sharding, trade-offs of each approach
Theory	CAP theorem — what it actually means for real design decisions
Architecture	Microservices vs monolith — microservices are a scaling solution, not a starting point
Message Queues	RabbitMQ/Kafka at a conceptual level
Study	Designing Data-Intensive Applications by Kleppmann — Chapters 1 and 2 this week
Practice	URL shortener, rate limiter, notification system — design on paper

Week 21 DevOps + Deployment

Docker	Images, containers, volumes, networks, docker-compose for local dev — write a real Dockerfile
CI/CD	GitHub Actions for automatic testing and deployment
Deployment	Deploy to Railway/Render or a DigitalOcean droplet for the full experience
Monitoring	Set up a basic uptime/alert monitoring
Mindset	'It works on my machine' is not professional engineering

Week 22 — Backend Capstone Project

Project	Real-time collaborative note-taking API
Features	User auth, PostgreSQL + Prisma, Redis caching, WebSocket live updates, BullMQ for email notifications
Quality	Proper error handling, Zod input validation, pino logging, CI/CD pipeline
README	Architecture decisions documented — explain why you made each choice
Interview Value	This project demonstrates: auth, real-time, caching, queues, deployment — in one place

PHASE 4 AI Integration + Final Capstone *Weeks 23–27 | ~5–6 hrs/day*

This is what makes your profile stand out in the current market. The goal is AI engineering, not just AI API usage.

Week 23 AI/ML Fundamentals for Software Engineers

Concepts	How LLMs work: tokens, embeddings, transformer architecture, attention mechanism, context windows
OpenAI API	Chat completions, system/user/assistant roles, temperature, token limits, API patterns
Embeddings	Numerical representations of meaning — similar meanings have similar vectors (cosine similarity)
Key Distinction	An engineer who understands AI builds AI features. An engineer who doesn't just calls APIs.

Week 24 Practical AI Integration — RAG Systems

Tools	LangChain.js or Vercel AI SDK for structured AI application development
Focus	RAG (Retrieval-Augmented Generation) — one of the most common AI engineering tasks right now
RAG Pipeline	Document chunking → embedding generation → vector storage → similarity search → context injection → LLM response
Vector DB	Pinecone or pgvector with PostgreSQL
Project	Document Q&A system: users upload docs and ask questions, AI answers with citations

Week 25 AI Feature Integration in Full-Stack Apps

Streaming	Streaming LLM responses via SSE or WebSockets — streaming UI in React with Vercel AI SDK
Prompt Engineering	System prompts, few-shot examples, chain-of-thought, structured output formats
Tool Use / Function Calling	How AI agents interact with external systems — this is how AI does real things
Project	Add an AI feature to your existing full-stack project: smart search, summarizer, or writing assistant

Weeks 26–27 — Final Capstone — Flagship AI SaaS Project

Project	AI-powered knowledge base SaaS — teams upload documents, chat interface, cited streamed answers
Features	User auth, subscription management, usage limits, admin dashboard, real-time chat
Frontend	Next.js App Router + TypeScript + Tailwind + Zustand + TanStack Query
Backend	Node.js + Express/Next.js API routes + PostgreSQL + Prisma + Redis + pgvector
AI	OpenAI API (or open-source LLM) with RAG, streaming responses, tool calling
DevOps	Docker + GitHub Actions CI/CD, deployed and live
README	Architecture diagram, tech choices, trade-offs explained — interview gold

What to Do Alongside This Roadmap

GitHub Discipline	Push code every single day. Your GitHub activity graph is a visual resume. Write meaningful commit messages. Every project needs a proper README.
Weekly Documentation	After every week, write a short technical summary of what you learned. Forces active recall and gives you material for interviews and resume.
Behavioral Interviews	After Week 6: 2–3 behavioral interview sessions weekly using the STAR method (Situation, Task, Action, Result).
Mock Technical Interviews	After Week 10: 1 mock technical interview per week. By Week 20, full mock interviews regularly.
Resume Updates	Update your resume at the end of each phase — not all at once at the end.
Reading	Designing Data-Intensive Applications (Kleppmann) alongside Phase 3. You Don't Know JS (Kyle Simpson) alongside Phase 1.

Technology Stack Summary

Layer	Technologies	Purpose
Languages	JavaScript (ES2024) + TypeScript	Core foundation — both in balance
Frontend	React, Next.js (App Router)	Industry-standard React stack
Styling	Tailwind CSS, CSS Modules	Utility-first, scalable styling
State Mgmt	Zustand, TanStack Query	Client + server state management
Backend	Node.js, Express.js	Server runtime and HTTP framework
Database	PostgreSQL, Prisma ORM	Relational data with type safety
Caching	Redis, pgvector	Performance + AI vector search
AI	OpenAI API, LangChain.js, Vercel AI SDK	LLM integration and RAG systems
DevOps	Docker, GitHub Actions, Vercel	Containerization and CI/CD
Testing	Vitest, React Testing Library, Playwright	Unit, component, and E2E testing

The Most Important Thing

Six months from now, you will not be the same person who started.

Treat every single day as non-negotiable. The frustration you feel now is energy — put it here.

You have everything you need. Now it is just about execution.